Protocol Emulator 1.0: Requirements for the Micro-architecture

Status: **requirements derived from the frozen Architecture 1.0** (27 Oct 2026). This is not a micro-architecture: it states what each block must do, how it must behave cycle by cycle where that is architecturally visible, and what budget it has. How to build it (pipelining, encodings of internal signals, cell choices, floorplan) is for the micro-architecture specification that follows.

Contents

0. How to read this document
1. Global requirements
2. Hierarchy and top-level interfaces
3. Pin layer
4. Host link
5. System bus and address decode
6. System controller
7. Main core
8. SRAM interface
9. CRC unit
10. Sub-core
11. Sideband interconnect
12. Cross-block timing summary
13. Area budgets
14. Verification requirements
15. Open points for the micro-architecture

0. How to read this document

- **Golden references.** `architecture-1.0.md` defines *what* the chip is. The reference models in `pe_arch1_model.zip` define cycle behaviour: `pe.py` for the sub-cores and pin-layer timing, `mc.py` for the main core and CRC unit. For every legal program the RTL **shall** produce the same pad values and FIFO words on the same clock edges as the model.
- **Requirement IDs.** `G-` global, `TOP-` top level, `PL-` pin layer, `HL-` host link, `BUS-` system bus, `SYS-` system controller, `MC-` main core, `MEM-` SRAM interface, `CRC-` CRC unit, `SC-` sub-core (with a sub-block tag), `SB-` sideband, `VER-` verification.
- **Shall** is mandatory. **Should** is a strong recommendation that the micro-architecture may override with a recorded reason. **Open** marks a point the architecture leaves to the micro-architecture; all open points are collected in §15 with a recommendation.
- **Pseudo-code** blocks are normative behaviour, written per clock: values on the right are register values at the start of the clock, assignments take effect at the clock edge that ends it.
- **Not in the reference model** (normative here anyway): host link, system bus, RUN_MASK, CTRL.STEP, breakpoint, pin-layer filter and loopback, GPIO, FA_OVR, debug registers.

1. Global requirements

- **G-01 One clock domain.** Every flop is clocked by `clk` on its rising edge. Latch enables are pulses derived from `clk` through clock-gating cells. No other clock exists; the host SPI clock is oversampled (§4).
- **G-02 Reset.** `rst_n` asserts asynchronously and is released synchronously to `clk` through a reset synchroniser. After reset: main core and all sub-cores halted, every program and configuration latch cleared to 0, every FIFO empty, every pad 'none' (uo pads drive 0, uio pads released), all bus-visible registers at their stated reset values.
- **G-03 Determinism.** A running sub-core advances one state per clock and is never stalled by bus traffic, host activity, the main core or another sub-core. Its pad behaviour depends only on its program, configuration, inputs and FIFO contents.
- **G-04 Latency contract.** At the TT pins: a state's pin action reaches its pad 1 clock after the state is entered; any event reaches a pin in 3 clocks; pad to pad is 5 clocks (with input filters at 0). These numbers are exact, not maxima.

- **G-05 Clock target.** 50 MHz (20 ns) at the slow corner after place and route, with all functions. 48 MHz and 40 MHz operation are the same logic at a lower clock.
- **G-06 Storage technology.** Sub-core programs and all configuration (sub-core, pin layer) are held in resettable latches (dlhrq-class), written only through clock-gated enables. FIFOs, shifters, counters, timers, state and pipeline registers are flops. The main core's program and data use the single SRAM macro. No other memory macro.
- **G-07 Programmability rule.** Program and configuration of a sub-core can be written only while that sub-core is halted. A write while it runs **shall** be ignored (not partially applied).
- **G-08 TT interface.** Top-level ports are the Tiny Tapeout user-module ports: `ui_in[7:0]`, `uo_out[7:0]`, `uio_in[7:0]`, `uio_out[7:0]`, `uio_oe[7:0]`, `ena`, `clk`, `rst_n`. `ena` is not used for function.
- **G-09 No combinational paths** from a TT input to a TT output. Every sub-core decision passes the registers that define the latency contract.
- **G-10 Parameters.** FIFO A depth (4) and the number of pin patterns (5) **should** be RTL parameters, because the architecture's fallback levers (§10 of the architecture) change them.
- **G-11 Area.** Each block has a budget (§13). The micro-architecture shall report its estimate against it before RTL starts.

2. Hierarchy and top-level interfaces

```
tt_um_protocol_emulator
├── rst_sync                  reset synchroniser (G-02)
├── host_link                 SPI target, bus master 0                    §4
├── sysbus                    arbiter, address decode, read mux           §5
├── main_core                 16-bit RISC, bus master 1                   §7
│   └── sram_if               RM_IHPSG13_1P_512x16 macro and its port     §8
├── sys_ctrl                  ID, SYS_CTRL, RUN_MASK, IRQ, doorbell,
│                             EVENTS, prescaler, CRC unit                 §6, §9
├── sc_pair[0..1]             elastic pair (SC0+SC1, SC2+SC3)             §10.3
│   └── subcore[0..1]         window, program bank, sequencer, action
│                             decode, datapath, output stage              §10
├── pin_layer                 15 output selects, 16 input chains, GPIO    §3
└── sideband                  SIG lines, WAIT vector, run/halt strobes     §11
```

Interface	From → To	Content
Bus	host_link, main_core → sysbus → all register blocks	16-bit word address, 16-bit write data, write/read strobes, 16-bit read data (registered), wait
IN, IN_R	pin_layer → every sub-core, sys_ctrl (IN register)	16 bits each; IN_R is IN one clock later
Outputs	subcore n → pin_layer	O0–O5 value and drive enable (registered), 24 pairs in total
Taps	subcore 0, 2 → pin_layer	8 values each + 1 shared drive enable (registered)
Host pins	TT pins ↔ host_link	ui0 SCK, ui1 CS_n, ui2 MOSI → host_link; MISO → uo0
SIG	each subcore → all subcores	2 one-clock pulses per sub-core, 8 lines
Status	subcores, CRC, doorbell → sys_ctrl	FIFO A has space, FIFO B has data, notify pending, halted or error, CRC idle
Run control	sys_ctrl → subcores	same-clock run strobes and halt strobes (RUN_MASK)
Prescaler ticks	sys_ctrl → subcores	3 one-clock pulses: ÷16, ÷256, ÷4096
IRQ	sys_ctrl → pin_layer	host IRQ level (pad source 2)

3. Pin layer

Purpose. Connects the 24 logical sub-core outputs, the 16 SA taps, GPIO and the host IRQ to 15 output pads, and conditions the 16 input pads into the shared IN and IN_R vectors.

3.1 Output pads

- **PL-01** There are 15 output-capable pads, numbered p = 0–14: uo1–uo7 (p = 0–6) and uio0–uio7 (p = 7–14). uo0 is the host link's MISO and is not selectable.
- **PL-02** Each pad has a PAD_CFG register at 0x1800 + p: `[5:0] SRC`, `[7:6] MODE` (0 push-pull, 1 open-drain, 2 input, 3 reserved = input).
- **PL-03** Source codes:

SRC	Source	Value	Drive enable
0	none	—	—
1	GPIO	GPIO_OUT[p]	GPIO_OE[p]
2	Host IRQ	IRQ level	1
3–7	reserved = none		
8 + 6n + k	Sub-core n output Ok (n 0–3, k 0–5)	its value register	its drive-enable register
32 + j	SC0 SA tap j (j 0–7)	SC0 tap j register	SC0 O5 drive-enable register
40 + j	SC2 SA tap j	SC2 tap j register	SC2 O5 drive-enable register
48–63	reserved = none		

- **PL-04** Pad drive, given the selected value v and drive enable e:

Pad	MODE	Pad output	Pad output enable
uio	push-pull	v	e
uio	open-drain	0	e AND NOT v
uio	input, or SRC = none	0	0
uo	push-pull or open-drain	v (no output enable exists on uo pads)	—
uo	input, or SRC = none	0	—

- **PL-05** The output path from the source registers to `uo_out` / `uio_out` / `uio_oe` is combinational (select and mode logic only). Adding a register would break the latency contract (G-04).
- **PL-06** One source may drive any number of pads. A pad has exactly one source.
- **PL-07** A PAD_CFG write takes effect on the next clock. Software maps pads before starting sub-cores (bring-up step 4); changing a live pad's source may glitch it for one clock, which is acceptable.

3.2 Input conditioning

- **PL-08** The 16 inputs are IN index 0–7 = `ui_in[0..7]`, 8–15 = `uio_in[0..7]`. uio inputs are taken from `uio_in` regardless of `uio_oe`, so a pad being driven is still read back.
- **PL-09** Each input passes a two-flop synchroniser. The first flop samples the pad directly, with no logic in front of it other than the loopback mux (PL-11).
- **PL-10 Glitch filter.** FILT_UI (0x1813) and FILT_UIO (0x1814) hold 2 bits per input. With f = 0 the filter is a wire. With f = 1–3 the filtered value changes only after the synchronised input has held the new level for f + 1 consecutive clocks, adding f clocks of latency to every use of that input. Reset value 0.
- **PL-11 Loopback.** LOOPBACK (0x1815) has one bit per input. When set, input k reads the output side of its pad instead of the pad: ui k ← the value driven on uo k (ui0 ← uo0 MISO); uio k ← its own pad output when its drive enable is 1, and 1 when released (the board pull-up). The loopback path **should** have the same latency as the pad path (place the mux in front of the synchroniser).

- **PL-12** IN is the 16-bit output of the filter stage. IN_R is IN delayed by one flop. Both go to every sub-core. With filters at 0, a pad level first sampled at edge k is in IN after edge k+1 and in IN_R after edge k+2 (this is what makes pad-to-pad 5 clocks).
- **PL-13** The input chain does not need a reset. Edges seen while it settles after `rst_n` are harmless: every sub-core is halted far longer than 3 clocks, and SOFT_RESET clears the event register.
- **PL-14** The host link's pins ui0–ui2 are synchronised for the host link independently of FILT and LOOPBACK (FILT/LOOPBACK settings never affect the host link).

3.3 GPIO and registers

Offset	Register	Reset	Notes
0x00–0x0E	PAD_CFG[p]	0	latches (configuration)
0x10	GPIO_OUT [14:0]	0	runtime data: flops
0x11	GPIO_OE [14:0]	0	runtime data: flops
0x12	IN [15:0]	—	read-only, the synchronised inputs
0x13	FILT_UI [15:0]	0	2 bits per ui input
0x14	FILT_UIO [15:0]	0	2 bits per uio input
0x15	LOOPBACK [15:0]	0	

- **PL-15** Unused offsets read 0 and ignore writes. GPIO registers are written by the bus at any time.
- **PL-16** Budget: 26.3k µm² (15 × 43:1 selects, 16 input chains, 16 tap paths, registers).

---

4. Host link

Purpose. Lets an external host (the demo board's microcontroller) read and write every bus address over SPI at any time.

- **HL-01** SPI target, mode 0 (CPOL = 0, CPHA = 0), on ui0 SCK, ui1 CS_n, ui2 MOSI, uo0 MISO.
- **HL-02** SCK, CS_n and MOSI are oversampled in the `clk` domain through synchronisers (G-01). The maximum SCK follows from that design (open O-01); the target is at least clk/10 (5 MHz at 50 MHz).
- **HL-03** A transaction is: CS_n low, a command (read or write), a 16-bit word address, then any number of 16-bit data words, CS_n high. The address auto-increments after each word. The exact bit layout, any status byte and the read turnaround are open (O-01).
- **HL-04** Each data word is one bus access by master 0. The host link has priority over the main core on every clock (BUS-03).
- **HL-05** Reads return the bus read data of the addressed word. A read with a side effect (FIFO B pop) is issued exactly once per word read, even if the host aborts after the word starts; no speculative reads of side-effect addresses.
- **HL-06** CS_n high at any time ends the transaction; an incomplete word is discarded (no partial write).
- **HL-07** After reset the host link is idle and fully functional: it is how everything else is loaded.
- **HL-08** MISO drives uo0 at all times (0 when idle).
- **HL-09** Budget: inside the 14k µm² for host link, bus and system controller.

---

5. System bus and address decode

- **BUS-01** 16-bit data, 16-bit word address. Two masters: host link (0), main core (1).
- **BUS-02** A write completes in 1 clock; a read returns registered data in 2 clocks.
- **BUS-03** The host link wins any clock in which both masters request. The main core waits; it has no deadline.
- **BUS-04** Address map:

Range	Target
0x0000–0x01FF	SRAM (shared with main-core instruction fetch, §8)
0x1000–0x11FF	Sub-core windows: n = (a − 0x1000) >> 7, offset = a & 0x7F
0x1800–0x181F	Pin layer
0x1C00–0x1C1F	System controller, CRC unit at 0x1C10–0x1C15
everything else	unmapped: reads 0, writes ignored

- **BUS-05** Reads with side effects (FIFO B pop, sub-core offset 0x06) happen once per bus read.
- **BUS-06** A target may hold the bus (wait): the CRC unit for a DATA write or VAL read while busy (CRC-04). Sub-core windows and the pin layer never wait.

6. System controller

Offset	Register	Behaviour	Reset
0x00	ID (R)	Constant design identifier (value open, O-05)	—
0x01	SYS_CTRL (W)	[0] main core run, [1] main core halt, [2] restart prescaler	—
0x02	RUN_MASK (W)	[3:0] run these sub-cores, [7:4] halt these	—
0x03	IRQ	Host IRQ status; set by the main core's IRQ instruction, cleared by writing 1	0
0x04	DOORBELL	Set by a host write, cleared by a main-core write; WAIT bit 12	0
0x05	EVENTS (R)	The live WAIT vector (SYS-05)	—
0x10–0x15	CRC unit	§9	

- **SYS-01 Reset sequence.** After `rst_n` the main core and all sub-cores are halted and the pin layer is at reset. Nothing runs until the host writes SYS_CTRL or RUN_MASK.
- **SYS-02 Same-clock start.** A RUN_MASK write makes every selected sub-core leave the halted state on the same clock edge; the sub-cores' first executed states are on the same clock. Halt bits work the same way.
- **SYS-03 Prescaler.** One free-running 12-bit counter shared by all sub-cores. It produces one-clock tick pulses when its low 4, 8 or 12 bits are all zero (÷16, ÷256, ÷4096). SYS_CTRL[2] clears it. It runs from reset.
- **SYS-04 IRQ.** The IRQ level is also pad source 2.
- **SYS-05 WAIT vector.** `[3:0]` FIFO A of sub-core n has space (count < depth); `[7:4]` FIFO B of sub-core n is not empty; `[11:8]` sub-core n has any notify flag set; `[12]` doorbell; `[13]` CRC unit idle; `[14]` any sub-core halted or with an error flag set; `[15]` 0. All bits are levels, computed every clock.
- **SYS-06** Budget: inside the 14k µm² for host link, bus and system controller (CRC unit counted with the main core).

7. Main core

Purpose. 16-bit RISC that frames and unframes data, computes CRC, moves words between SRAM and the sub-core FIFOs, and recovers from protocol errors.

- **MC-01 Registers.** r0 reads 0 and ignores writes; r1–r7 are general purpose; r7 is the link register for JAL. 16-bit PC.
- **MC-02 Instruction set** exactly as architecture §5.1 and `mc.py`:

Op [15:12]	Format	Behaviour
0	rd [11:9], rs [8:6], rt [5:3], fn [2:0]	rd = rs op rt: ADD, SUB, AND, OR, XOR, SLL, SRL, SRA (shift by rt[3:0]; SRA arithmetic)
1	rd, rs, imm6	ADDI: rd = rs + sign-extended imm6

Op [15:12]	Format	Behaviour
2	rd, rs, type [5:4], amt [3:0]	SHI: rd = rs SLL/SRL/SRA amt
3	rd, rs, imm6	ANDI: rd = rs AND zero-extended imm6
4	rd, rs, uimm6	LD: rd = mem[rs + uimm6]
5	rd, rs, uimm6	ST: mem[rs + uimm6] = rd
6, 7, D	rd, rs, off6	BEQ, BNE, BLTU (unsigned): if true, PC = PC + 1 + sign-extended off6
8	rd, simm9	LI: rd = sign-extended simm9
9	rd, imm8	LIH: rd = imm8 << 8 | rd[7:0]
A, B	off12	JMP; JAL (r7 = PC + 1): PC = PC + 1 + sign-extended off12
C	rd, rs, fn [5:0]	fn 0 JR rs; 1 WAIT rd, rs; 2 HALT; 3 IRQ; 4 NOP
E, F	—	Reserved: execute as NOP (as the reference model does)

- **MC-03 Timing.** One instruction per clock, plus one clock for: a load or store to SRAM (the fetch port is shared), a bus read (registered data), a taken branch, JMP, JAL and JR. A host-link access to SRAM in the same clock adds a clock (BUS-03).
- **MC-04 WAIT.** `WAIT rd, rs` holds the PC until (EVENTS & rs) ≠ 0, then writes the hit bits (EVENTS & rs) to rd and continues. WAIT with rs = 0 sleeps until the main core is halted or reset.
- **MC-05 Reset and control.** After reset the main core is halted with PC = 0. SYS_CTRL[0] starts it at its current PC; SYS_CTRL[1] and the HALT instruction halt it.
- **MC-06** Instruction fetch addresses are 0x0000–0x01FF; behaviour for a PC beyond 0x01FF is open (O-06).
- **MC-07** Load/store addresses are 16-bit and reach the whole bus map (BUS-04).
- **MC-08** Budget: 32k µm² including the CRC unit.

8. SRAM interface

- **MEM-01** One RM_IHPSG13_1P_512×16 macro (the CMOS5L SRAM library variant; see the physical-design plan). 512 × 16 bits, single port.
- **MEM-02** One access per clock: main-core fetch, main-core data or host-link access, arbitrated host > main-core data > fetch.
- **MEM-03** Read data is used in the clock after the access (the macro's clock-to-output, 3.7–3.9 ns typical, 6.3–6.6 ns slow, is the start of the main core's critical path).
- **MEM-04** Contents are undefined after power-up. The host loads code and data before starting the main core.
- **MEM-05** Macro control pins are driven so that no write-through or BIST mode is entered by accident (open O-09 for the exact pin use).

9. CRC unit

Address	Register	Behaviour
0x1C10 / 0x1C11	POLY_L / POLY_H	Polynomial bits [15:0] / [31:16]; reflected form when CFG.reflect = 1
0x1C12	CFG	[4:0] width − 1 (width 1–32), [5] reflect
0x1C13	DATA	Write a byte: the unit processes its 8 bits, 1 bit per clock
0x1C14 / 0x1C15	VAL_L / VAL_H	Write the initial value; read the result

- **CRC-01** Per bit, MSB-first mode (data bits taken from DATA[7] down to DATA[0]): `fb = VAL[W−1] ^ bit;` `VAL = ((VAL << 1) & mask(W)) ^ (fb ? POLY : 0)`.

- **CRC-02** Reflected mode (data bits from DATA[0] up): `fb = VAL[0] ^ bit; VAL = (VAL >> 1) ^ (fb ? POLY : 0)`.
- **CRC-03** A DATA write makes the unit busy for 8 clocks. Busy is WAIT bit 13 (inverted: bit 13 = idle).
- **CRC-04** A DATA write or a VAL read issued while busy waits until the unit is idle (bus wait, BUS-06). How a host-link access to the CRC behaves while busy is open (O-07).
- **CRC-05** Check values that the RTL shall reproduce: CRC-8/SMBUS 0xF4, CRC-16/ARC 0xBB3D, CRC-32 0xCBF43926 (with the initial value and final XOR applied by software as in the reference bench).

---

10. Sub-core

The sub-core is specified as the blocks below. Everything in this section is per sub-core unless it names the pair.

10.1 General

- **SC-01** One state per clock; no stall source exists (G-03).
- **SC-02** "Running" means not halted. While halted: the state holds, no pin action or micro-op fires, but the datapath (timers, shifters, counter events, output following) keeps operating on its configuration.
- **SC-03** The reference behaviour is `pe.py SubCore.tick()`. Where this section and the model disagree, raise it; do not pick one silently.

10.2 Bus window

Offset	Register	Access	Behaviour
0x00	CTRL	W	[0] RUN, [1] HALT, [2] STEP, [3] SOFT_RESET, [4] FLUSH_A, [5] FLUSH_B, [6] CLR_ERR
0x01	STATUS	R	[4:0] state, [5] halted, [6] breakpoint hit, [9:7] NTF1–3, [12:10] FA_EMPTY, FB_OVR, FA_OVR
0x02	NTF ack	W	Writing 1 to bit k–1 clears NTFk
0x03	MAILBOX	RW	[3:0] MB0–MB3
0x04	FIFO A push	W	Push data word, tag 0
0x05	FIFO A push tagged	W	Push data word, tag 1
0x06	FIFO B pop	R	Pop; returns the data bits (0 if empty)
0x07	LEVELS	R	[3:0] FIFO A count, [7:4] FIFO B count, [8] FIFO B head tag
0x08–0x0F	Debug	R	§10.20
0x10–0x31	Configuration	W	34 words, §10.21; only while halted
0x40–0x5F	Program	W	Row s at 0x40 + 2s (bits 15:0), 0x41 + 2s (bits 31:16); only while halted

- **SC-WIN-01** Configuration and program writes are applied only while halted (G-07). For a pair partner whose rows are lent to the master (split 24 or 32), writes to the lent rows are applied only when the master is also halted.
- **SC-WIN-02** CTRL bits written together act in this order: CLR_ERR, FLUSH_A/FLUSH_B, SOFT_RESET, HALT, RUN, STEP. So `SOFT_RESET | RUN` restarts a halted sub-core from its entry state in one write.
- **SC-WIN-03** SOFT_RESET: state = ENTRY with the entry flag set; outputs to their initial source (§10.17); timers stopped with count 0 and wave 0; SA and SB cleared and disarmed; counter 0; event register, sticky bits and sticky EXT cleared; SIG outputs 0; both FIFOs emptied. It does **not** change halted/running, error flags, notify flags, mailbox or FBNEW.
- **SC-WIN-04** FLUSH_A / FLUSH_B empty that FIFO without touching anything else, also while running.

- **SC-WIN-05** STEP while halted runs exactly one sequencer clock (the state executes as if running for one clock, entry actions included if it is the state's first clock). STEP while running is ignored.
- **SC-WIN-06** Unused offsets read 0 and ignore writes. Configuration and program words are write-only (read-back is not required).

10.3 Program bank and elastic pair

- **SC-PRG-01** Each sub-core has 16 rows × 32 bits of resettable latches, written in 16-bit halves through clock-gated enables (one gate per 16-bit half-row).
- **SC-PRG-02** The bank is read every clock at the address of the current state; the read path is part of the next-state loop (§10.4).
- **SC-PRG-03 Pair modes** (SEQ_CFG.SPLIT of the master SC0 or SC2; the partner's SPLIT is ignored):

SPLIT	Master fetches	Partner fetches	Partner sequencer
0 (16/16)	own rows 0–15 for states 0–15	own rows 0–15	runs
1 (24/8)	own rows for states 0–15; partner rows 0–7 for states 16–23	own rows 8–15 (states 8–15)	runs
2 (32/0)	own rows for 0–15; partner rows 0–15 for states 16–31	—	held halted; its datapath and outputs idle

- **SC-PRG-04** In 24/8 both sub-cores read the partner bank on the same clock from disjoint halves; the partner bank therefore needs independent selects for its rows 0–7 (driven by the master) and rows 8–15 (driven by the partner).
- **SC-PRG-05** State bit 4 of the master selects own or partner bank data. A master state ≥ 24 in 24/8 mode, or a partner state < 8 in 24/8 mode, is a programming error; the hardware need not detect it.
- **SC-PRG-06** Budget: 22.2k µm² per sub-core (rows, row gates, read mux, half of the pair select).

10.4 Sequencer

- **SC-SEQ-01** State register 5 bits and an entry flag.
- **SC-SEQ-02** Next state, every running clock:

```
hit0 = PRE0.EN & (PRE0.ALWAYS | instr.PE) & pre_event(PRE0.EV)
hit1 = PRE1.EN & (PRE1.ALWAYS | instr.PE) & pre_event(PRE1.EV)
if   hit0: next = PRE0.VECTOR; entry_next = 1
elif hit1: next = PRE1.VECTOR; entry_next = 1
else:      next = (C[instr.COND] ? instr.T : instr.F) | (disp_fired ? disp_field : 0)
           entry_next = (next != state)
pre_event: E0–E3 = event register pulse, T1P/T2P = event register,
           ERR = error event, EXT = EXT level (not sticky)
```

- **SC-SEQ-03 Conditions** C[0..31] from registered values only:

C	Source
0–3	E[k] OR (EVUk.STICKY AND sticky[k])
4–7	T1P, T1C, T2P, T2C from the event register
8	SAD = NOT SA.valid
9	SBD = SB.done
10	CNT = 0
11	SBIT_SRC ? SB.last : SA lane-0 output value (§10.9)
12	MATCH (§10.13)
13	FIFO A not empty

C	Source
14	FIFO A not empty AND head tag = 1
15	EXT level OR (EXT.STICKY AND ext_sticky)
16–31	IN_R[0..15]

- **SC-SEQ-04 Sticky consumption.** When no preemption hits and C[COND] is true: if COND = k < 4 and EVUk.STICKY, clear sticky[k]; if COND = 15 and EXT.STICKY, clear ext_sticky.
- **SC-SEQ-05 Halt.** The HALT micro-op sets halted at the end of the clock; the state update of that clock still happens (the sub-core halts in the next state, with its entry flag as computed). CTRL.HALT and RUN_MASK halt take effect at the next clock edge.
- **SC-SEQ-06 Breakpoint.** With BP_EN, the sub-core halts when it enters BP_STATE and sets STATUS[6]. The exact clock (before or after BP_STATE's actions) is open (O-03).
- **SC-SEQ-07 Timing.** The loop state → program read (incl. pair select) → COND decode → 32:1 condition mux → T/F select → DISP OR → preemption muxes → state register closes in one clock at G-05.
- **SC-SEQ-08** Budget: 4.3k µm².

10.5 Action decode

- **SC-ACT-01** Actions are valid only in a running clock with the entry flag set.
- **SC-ACT-02** Pin action → per-output codes `code[0..5]` (0 = none):

```
OSEL 0–5 : code[OSEL] = POP
OSEL 6   : code[k] = POP for k < min(SA.LANES, 6)
OSEL 7   : POP 1–5 -> code[k] = PATk_code[k] for all k
           POP 6   -> code[k] = 3 (release) for all k
           POP 7   -> code[k] = 7 (idle)    for all k
           POP 0   -> nothing
```

- **SC-ACT-03** OPA and OPB each decode to one of 32 micro-op strobes; both strobes are active together. Where two strobes target the same resource, priorities are: SA: LOAD > STOP > STEP; SB: ARM > PUSH > STOP > STEP; CNT: LDP > LDF > LDB > (INC, DEC). These priorities also apply against hardware events (a shift event in the clock of an SA.LOAD is ignored).
- **SC-ACT-04** Micro-op effects are those of architecture §4.5; per-block behaviour follows in §10.6–§10.18.

10.6 Event units and event register

- **SC-EVU-01** Four units. Each computes, every clock:

```
cur = IN[PIN]; prev = IN_R[PIN]
e = MODE==level ? cur : MODE==rise ? cur & !prev : MODE==fall ? !cur & prev : cur ^ prev
if QEN: e &= (IN[QPIN] == QLVL)
```

- **SC-EVU-02** The event register captures, every clock (running or halted): E0–E3 from SC-EVU-01, T1P, T1C, T2P, T2C (§10.7), and ERR (an error occurred this clock, whether or not its flag was already set).
- **SC-EVU-03** Sticky bits: `sticky[k] = (sticky[k] | E[k]) & !(consume[k] | CLR.Ek)` (clear wins).
- **SC-EVU-04** Budget: 3.8k µm².

10.7 Timers T1, T2

- **SC-TMR-01** Per timer, every clock (running or halted):

```
tick    = PSC==0 ? 1 : prescaler_tick[PSC]                // shared ticks, SYS-03
// ext: T1 fraction carry for this period (else 0)
mp      = run & tick & (count == PER + ext)
mc      = run & tick & (count == CMP)
// restart_event: E0–E3 or the other timer's P/C, from the event register
restart = GO strobe | (RST_EN & restart_event(RST_SEL))
burst_end = GATE & (cnt_next == 0) & (cnt != 0)      // counter reaches 0 (§10.8)
stop    = STOP strobe | (ONESHOT & mp) | burst_end
if restart:  count = RLD; run = 1; wave = 0
else:
    if stop: run = 0
    if mp:   count = 0; if T1: {ext, acc} = acc + FRAC   // carry: next period +1
    elif run_after_stop & tick: count = count + 1
    if mc: wave = 1
    if mp: wave = 0
P = mp; C = mc   -> event register
```

- **SC-TMR-02** The AUX clock CLKk = wave & !(GATE & CNT == 0).
- **SC-TMR-03** Only T1 has FRAC (8 bits) and its accumulator.
- **SC-TMR-04** Budget: 6.4k µm² for both timers (configuration counted in §10.21).

10.8 Counter

- **SC-CNT-01** 16 bits, every clock (running or halted):

```
if   LDP: cnt = PRESET
elif LDF: if FIFO A not empty: cnt = pop(FIFO A) else set FA_EMPTY
elif LDB: cnt = SB.sr
else:
    dec = DEC strobe | (DEC_EN & cnt_event(DEC_SEL))
    inc = INC strobe | (INC_EN & cnt_event(INC_SEL))
    if dec & !inc: cnt = WRAP ? cnt - 1 : max(cnt - 1, 0)        (16-bit)
    if inc & !dec: cnt = WRAP ? cnt + 1 : min(cnt + 1, 0xFFFF)
cnt_event: E0–E3, T1P, T2P from the event register;
           SASH = SA shifted this clock; SBCAP = SB captured this clock
```

- **SC-CNT-02** CAP pushes the counter value from the start of the clock into FIFO B.
- **SC-CNT-03** Budget: 3.6k µm².

10.9 SA (transmit shifter)

- **SC-SA-01** State: sr[15:0], bit count, valid, armed, skip, hold[7:0]. Every clock (running or halted):

```
sh       = event(SA.EV) from the event register
at_last  = valid & (bc + LANES >= N)
cur_grp  = valid ? group(sr) : hold
if   LOAD: armed = 1; if FIFO A not empty: sr = pop; bc = 0; valid = 1; skip = SKIP1
                      else: valid = 0
elif STOP: valid = 0; armed = 0
elif STEP | (sh & armed):
    if sh & skip & !STEP:                    skip = 0
    elif (!valid | at_last) & STREAM & armed:
        if FIFO A not empty: sr = pop; bc = 0; valid = 1; skip = 0  else: valid = 0
        shifted = at_last
    elif at_last:                            valid = 0; armed = 0; shifted = 1
    elif valid:
        sr = MSB ? sr << LANES : sr >> LANES; bc += LANES; shifted = 1
if !valid_next: hold = cur_grp
if SKIP strobe & !LOAD & FIFO A not empty: pop (discard)
```

- **SC-SA-02** Lane values: LSB-first, lane j = sr[j]; MSB-first, lane j = sr[N − 1 − j]. Output value of lane j = valid ? lane j : (UDF == hold ? hold[j] : UDF).
- **SC-SA-03** SA taps (SC0 and SC2): tap j = lane j value (j = 0–7), registered in the output stage (§10.17).
- **SC-SA-04** Budget: shifters 7.3k µm² for SA and SB together.

10.10 SB (receive shifter)

- **SC-SB-01** State: sr[15:0], bit count, armed, done, last. Every clock (running or halted):

```
done = STREAM ? 0 : done
if   ARM:  bc = 0; sr = 0; armed = 1; done = 0
elif PUSH: if bc > 0: push_B(MSB ? sr : sr >> (N - bc)); bc = 0; sr = 0; done = 0
elif STOP: armed = 0
elif STEP | (event(SB.EV) & armed):
    smp = IN_R[PIN + LANES - 1 : PIN]                    // lane i = IN_R[PIN + i]
    sr  = MSB ? ((sr << LANES) | smp) & mask(N) : (sr >> LANES) | (smp << (N - LANES))
    last = smp[0]; bc += LANES; captured = 1
    if bc >= N: done = 1; if STREAM: push_B(sr); bc = 0; sr = 0  else: armed = 0
```

- **SC-SB-02** PIN + LANES − 1 ≤ 15 (assembler rule; hardware behaviour beyond it is don't-care).

10.11 FIFO A

- **SC-FA-01** Depth 4 (G-10), 17 bits per entry (16 data + tag), flops.
- **SC-FA-02** Writes: bus offset 0x04 (tag 0) and 0x05 (tag 1). A write to a full FIFO is dropped and sets FA_OVR.
- **SC-FA-03** Reads (pops): SA.LOAD, SA stream reload, SA.SKIP, CNT.LDF. The reference model applies up to three pops in one clock in the order SA (load or reload), SKIP, LDF. Whether the RTL supports more than one pop per clock is open (O-02).
- **SC-FA-04** Exposes: not empty (TXNE), head tag (TXTAG), head data (DISP FAHEAD, 0 when empty), count (LEVELS, WAIT bit).
- **SC-FA-05** A bus write and pops in the same clock both take effect. Fullness for FA_OVR is judged at the start of the clock (a write to a FIFO that is full at the start of the clock is dropped even if it pops in that clock); software never relies on that case.

10.12 FIFO B

- **SC-FB-01** Depth 2, 17 bits per entry, flops.
- **SC-FB-02** Writers, in order within one clock: SB word (stream push or SB.PUSH), MARK (tag 1, the current state number in bits 4:0), CAP (counter value). Up to three writes per clock; each write that finds the FIFO full is dropped and sets FB_OVR. UART RX's `SB.PUSH, MARK` relies on two writes in one clock.
- **SC-FB-03** FBNEW: a 16-bit register holding the data of the newest word written (by any writer). It is not cleared by SOFT_RESET.
- **SC-FB-04** Read: bus offset 0x06 pops (data bits); LEVELS[8] shows the head tag before the pop. RXFULL = count == depth.
- **SC-FB-05** Budget: FIFO A and B together 9.7k µm².

10.13 Matcher

- **SC-MAT-01** MATCH = ((SB.sr ^ MATCH_VAL) & MATCH_MASK) == 0, combinational from registers. Budget 0.5k µm².

10.14 EXT

- **SC-EXT-01** `level = OR over enabled MASK bits of {E0, E1, E2, E3, T1P, T2P (event register), MAILBOX[MB_SEL], SIG[SIG_SEL] (registered SIG lines), RXFULL, ERR (any error flag)}`.

- **SC-EXT-02** `ext_sticky = (ext_sticky | level) & !(consume_ext | CLR.EXT)`.
- **SC-EXT-03** Budget 0.6k µm².

10.15 DISP

- **SC-DSP-01** `field = source[BASE + width − 1 : BASE]`, width 2 or 3, source ∈ {SB.sr, FBNEW, FIFO A head data (0 if empty), IN_R}. Bits above bit 15 read 0.
- **SC-DSP-02** When the DISP strobe is active, next = (C ? T : F) | field (SC-SEQ-02). The assembler guarantees T = F and alignment; the hardware just ORs.

10.16 Preemption

- **SC-PRE-01** Two entries, behaviour in SC-SEQ-02. A preemption does not cancel the current clock's actions (they fire if it is the state's entry clock) and does not consume sticky conditions.

10.17 Output stage, patterns and taps

- **SC-OUT-01** Per output k: mode ∈ {VAL, SA, AUX}, v, oe (internal), and registered q (value) and qoe (drive enable). Every clock (running or halted; codes are 0 when no action fires):

```
code 1: mode=VAL, v=0, oe=1      code 2: mode=VAL, v=1, oe=1      code 3: mode=VAL, oe=0
code 4: mode=VAL, v=!(q ^ INV), oe=1  (toggle the logical value)
code 5: mode=SA,  oe=1           code 6: mode=AUX, oe=1
code 7: initial source: IDLE -> VAL, v=IDLE, oe=1 | Z -> VAL, v=IDLE, oe=0
                        SA   -> SA, oe=1          | AUX -> AUX, oe=1
// SA and AUX values come from the current registers
value = mode==VAL ? v : mode==SA ? sa_lane(LANE) : aux(AUX)
q   = value ^ INV
qoe = (mode==SA & OE_LANE) ? sa_lane(LANE + 1) : oe
```

- **SC-OUT-02** AUX sources: CLK1, CLK2 (SC-TMR-02), MANCH = SA lane 0 ⊕ CLK1, NPART, SBBIT = SB.last, E0LVL = IN_R[EVU0.PIN], E1LVL = IN_R[EVU1.PIN], ZERO.
- **SC-OUT-03** NPART: output k's q = !(q_next of output k xor 1, as a logical value) ^ INV, computed in the same clock, so the pair changes on the same edge.
- **SC-OUT-04** Patterns: 5 × 6 pin codes of 3 bits (PATO registers). All six outputs take their codes on the same edge.
- **SC-OUT-05** Taps (SC0, SC2): 8 registered values = SA lanes 0–7 from current registers, and one registered drive enable = O5's next qoe. Same edge as the outputs.
- **SC-OUT-06** SOFT_RESET sets each output to its initial source (code 7 behaviour) and q/qoe accordingly.
- **SC-OUT-07** Budget: 6.7k µm² (configuration counted in §10.21).

10.18 Status, errors, notify, SIG, mailbox

- **SC-STS-01** Error flags FA_EMPTY, FB_OVR, FA_OVR: set by their causes, cleared only by CTRL.CLR_ERR. The ERR event (event register) is 1 in the clock after any error occurs.
- **SC-STS-02** NTF1–3 flags: set by NTF micro-ops, cleared by NTF ack. Any flag set drives WAIT bit [8 + n].
- **SC-STS-03** SIG0/SIG1 micro-ops drive this sub-core's SIG lines 2n / 2n + 1 high for exactly one clock through a register; all sub-cores (including this one) see them in their EXT the clock after.
- **SC-STS-04** MAILBOX bits are written by the bus and read by EXT (MB source) as levels.
- **SC-STS-05** Status outputs to sys_ctrl: FIFO A has space, FIFO B not empty, any notify, halted OR any error flag.

10.19 Breakpoint and step

- **SC-DBG-01** BP_EN/BP_STATE (SEQ_CFG) and CTRL.STEP as in SC-SEQ-06 and SC-WIN-05.

10.20 Debug read-back

- **SC-DBG-02** Offsets 0x08–0x0F are read-only views of: entry flag and sticky/EXT bits, CNT, T1 count, T2 count, SA shift register, SB shift register, FIFO A head word and tag, event register. Bit layout is open (O-04). Reading them has no side effects.

10.21 Configuration registers

- **SC-CFG-01** 447 bits in 34 field-aligned words at offsets 0x10–0x31, exactly the layout of architecture §4.11 (also Appendix A of the programming guide). Resettable latches cleared by `rst_n`; written only while halted.
- **SC-CFG-02** Configuration is static while running: the RTL may treat it as constant during operation (it changes only while the sub-core is halted).
- **SC-CFG-03** Budget: 12.6k µm² (447 latches and the reset buffering), plus 3.9k µm² for configuration and bus decode (66 word strobes, 8 debug words).

---

11. Sideband interconnect

- **SB-01 SIG lines.** 8 registered one-clock pulse lines: sub-core n drives lines 2n and 2n + 1; every sub-core receives all 8.
- **SB-02 Status.** Per sub-core: FIFO A has space, FIFO B has data, notify pending, halted or error, into the WAIT vector (SYS-05). Combinational from registers is acceptable; the WAIT vector feeds only the main core.
- **SB-03 Run control.** RUN_MASK run and halt strobes reach all four sub-cores on the same clock (SYS-02).
- **SB-04 Prescaler ticks.** The three tick pulses reach all sub-cores on the same clock.
- **SB-05** Sideband signals do not use the system bus and add no bus traffic.

---

12. Cross-block timing summary

Path	Registers on it	Clocks
Pad → sync1 → sync2/filter (IN) → IN_R	3 (filter 0)	pad level first sampled at edge k is in IN_R after k+2
IN, IN_R → event register	1	after k+2 (same edge as IN_R)
Event register / IN_R → condition → state register	1	after k+3
State (entry) → output register → pad (combinational pin layer)	1	after k+4
Timer count = PER → event register → state register → output register	3	
Timer count = CMP/PER → wave register → output register	2	
Micro-op → SA register → output register	2	
SIG micro-op → SIG register → other sub-core's EXT condition	2 to its state change	
Bus write → configuration latch	1	(only while halted)
Bus read → read data	2	

13. Area budgets

Block	Budget (k µm²)	Notes
Sub-core (each)	81.6	program 22.2, FIFOs 9.7, shifters 7.3, outputs 6.7, timers 6.4, sequencer 4.3, config decode 3.9, event units 3.8, counter 3.6, matcher and EXT 1.1, configuration latches 12.6
Pin layer	26.3	
Main core with CRC	32.0	
Host link, bus, system controller	14.0	
Subtotal	398.5	
With +10 % CTS, buffers, hold fixing	438.4	50.3 % density on 6×4, 37.2 % on 8×4
SRAM macro	45.3	excluded from density

The budgets are scaled estimates (±15 %). The micro-architecture reports per-block estimates against them; synthesis results replace them as soon as they exist.

14. Verification requirements

- **VER-01 Cycle equivalence.** For every directed and random test, the RTL's per-clock pad values (value and drive enable) and FIFO B words match `pe.py` / `mc.py` exactly. `wave.py` VCDs from the model are the comparison reference.
- **VER-02 Directed tests.** The 12 programs of the architecture §7 with their benches, the pin tests and the system test (main core + sub-core + CRC) pass in RTL simulation and in gate-level simulation.
- **VER-03 Random programs.** Randomly generated legal programs and configurations, run against the model, cover every condition, micro-op, pin code, pattern, DISP source and width, preemption mode, split mode and shifter mode.
- **VER-04 Register tests.** Every register's reset value, write and read behaviour, the halted-only write rule (G-07, SC-WIN-01) and CTRL bit ordering (SC-WIN-02).
- **VER-05 Corner cases.** Simultaneous micro-ops on one resource, FIFO full/empty in every combination, three FIFO B writes in one clock, STEP and breakpoint, HALT followed by RUN, soft reset during a transfer, pair split changes.
- **VER-06 Reset.** After `rst_n`, no pad moves until software configures it; latches read back (through behaviour) as cleared.
- **VER-07 Host link.** Burst read/write at the maximum SCK, abort mid-word, FIFO pop exactly once per read.

15. Open points for the micro-architecture

These are left open by the architecture. Each has a recommendation; the decision is recorded in the micro-architecture specification and does not change the architecture.

ID	Open point	Recommendation
O-01	Host link frame format: command encoding, read turnaround/dummy bits, bit order, maximum SCK	MSB-first; one command byte (R/W + reserved bits), 16-bit address, 16-bit words; one dummy word before read data; SCK ≤ clk/10 guaranteed
O-02	FIFO A pops per clock (the model allows up to three: SA, SKIP, LDF)	Support one pop per clock and make the assembler reject two FIFO A consumers in one state; document stream reload + CNT.LDF/SA.SKIP in the same clock as unsupported. Confirm no frozen example program needs more (none does)

ID	Open point	Recommendation
O-03	Breakpoint clock	Halt on entering BP_STATE before its actions fire; STEP then executes it
O-04	Debug register bit layout	One register per item listed in SC-DBG-02, LSB-aligned
O-05	ID register value	A constant identifying the design and the architecture version (1.0)
O-06	Main-core PC beyond 0x01FF	Wrap modulo 512
O-07	Host access to the CRC unit while busy	Bus wait, as for the main core
O-08	Loopback mux position	In front of the synchroniser (equal latency, PL-11)
O-09	SRAM macro control pins (read enable, write enable, BIST, write-through)	Never assert write and read enable together; tie BIST inputs inactive; confirm from the macro's datasheet and Liberty file
O-10	Reset of the input synchronisers and filters	No reset needed (PL-13)
O-11	Write-ignore implementation for configuration while running	Gate the latch-enable clock gates with `halted`